OOP Assignment #2 - Car Marketplace System

Name: Izaan Aamir | Roll No: 25K-0059 | Section: BAI-2A

Introduction

This assignment extends the Car Marketplace System from Assignment 01 by adding Inheritance, Polymorphism, Abstraction, Operator Overloading, and Friend Functions.

The base classes from A1 are kept the same. New classes and features are added on top:

- 3 abstract classes with pure virtual functions (display, filter, user)
- Car and Bike as derived classes of Vehicle
- Bidding system - buyers can place bids on approved listings
- Seller review system - buyers can rate sellers out of 5 stars
- Advanced search - filter by type, brand/model, and price range
- Operator overloads on Vehicle, Bid, Review and Message
- Friend functions to access private data where needed

1. Abstract Classes

1.1 display

Forces any vehicle to implement displayDetails() and getCategory(). This makes Car and Bike display differently using the same function call.

```
class display {
public:
    virtual void displayDetails() const = 0;
    virtual string getCategory() const = 0;
    virtual ~display() {}
};
```

1.2 filter

Forces any vehicle to implement matchesFilter(). Car also checks bodyStyle when filtering, while base Vehicle only checks brand and model.

```
class filter {
public:
    virtual bool matchesFilter(string key) const = 0;
    virtual ~filter() {}
};
```

1.3 user

Forces every user type (Buyer, Seller, Admin) to implement showPanel() and showProfile(). This is used in main() with a User* array to demonstrate polymorphism.

```
class user {
public:
    virtual void showPanel() = 0;
    virtual void showProfile() const = 0;
    virtual ~user() {}
};
```

2. Inheritance

6 inheritance relationships are implemented:

#	Relationship	Reason
1	user -> User	User must implement showPanel and showProfile, different for each role
2	display -> Vehicle	Every vehicle listing must be displayable
3	filter -> Vehicle	Every vehicle must support keyword filtering
4	Vehicle -> Car	Car adds body style and door count
5	Vehicle -> Bike	Bike adds style and CC
6	User -> Buyer / Seller / Admin	Three roles share name and email but work differently

3. Polymorphism

Function Overriding

displayDetails() is overridden in Car and Bike. matchesFilter() is overridden in Car to also check bodyStyle. showPanel() and showProfile() are overridden in all three user types.

```
User* users[3];
users[0] = &admin;
users[1] = &seller;
users[2] = &buyer;

for (int i = 0; i < 3; i++)
    users[i]->showProfile();
```

Function Overloading

All classes have both a default constructor and a parameterised constructor. This is the same pattern used in A1.

4. Operator Overloading

Class	Operator	Reason
Vehicle	==	Compare brand, model, year - detect duplicate listings
Vehicle	+	Add prices of two vehicles together for bundle display
Bid	==	Check if same buyer already placed a bid
Bid	>	Compare bid amounts to find the highest bid
Review	==	Check if same buyer already reviewed this seller
Message	==	Check if identical message already exists

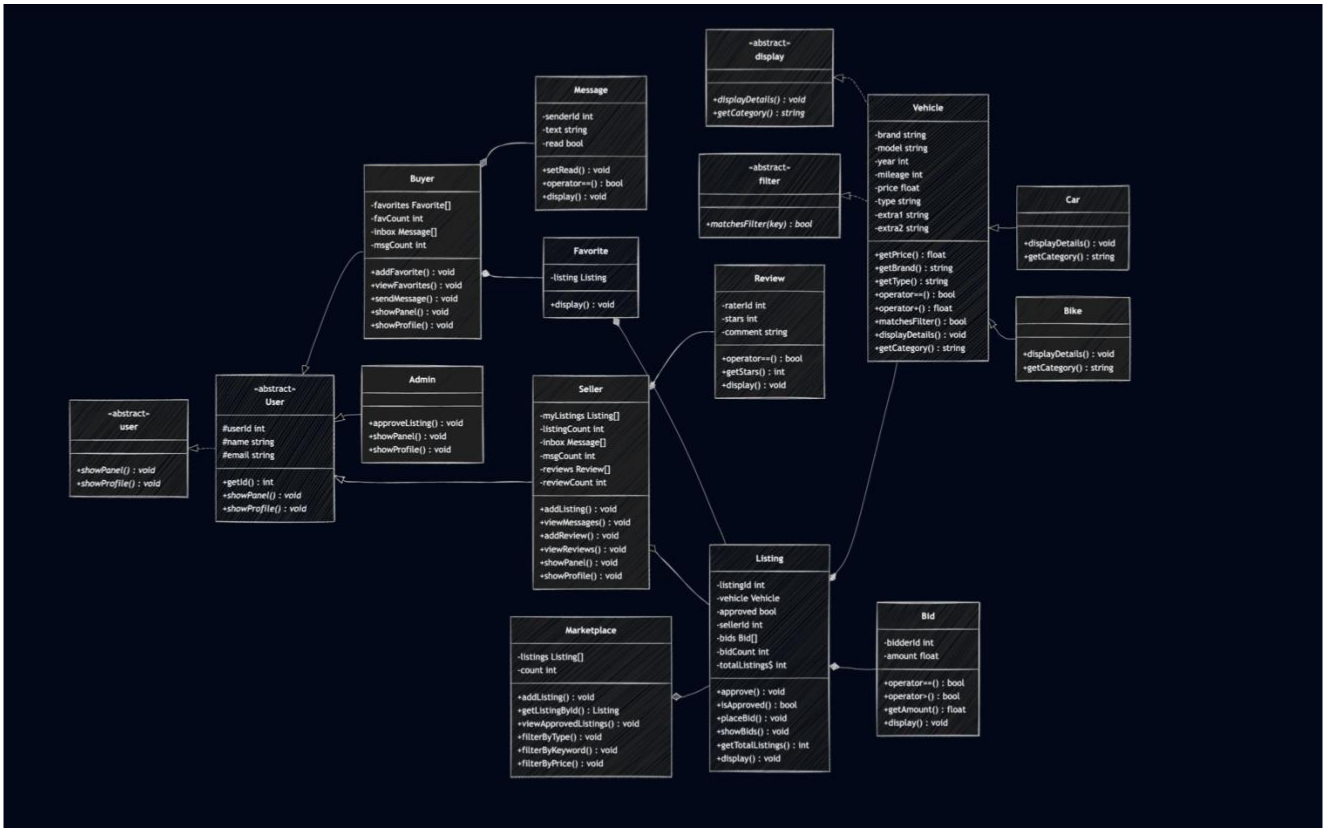
```
void placeBid(int buyerId, float amt) {
    Bid b(buyerId, amt);
    for (int i = 0; i < bidCount; i++)
        if (bids[i] == b) {
            cout << "Already placed a bid.\n";
            return;
        }
    bids[bidCount++] = b;
}
```

5. Friend Functions

Friend	Class	Reason
calcAvgRating(arr, count)	Review	Needs private stars field to compute average rating
printInbox(arr, count)	Message	Needs private text and read fields to print messages
showPlatformSummary(market)	Marketplace	Needs private listings[] and count for admin analytics
friend class Marketplace	Vehicle	filterByPrice() accesses private price field directly

```
float calcAvgRating(Review arr[], int count) {
    float total = 0;
    for (int i = 0; i < count; i++)
        total += arr[i].stars;
    return total / count;
}
```

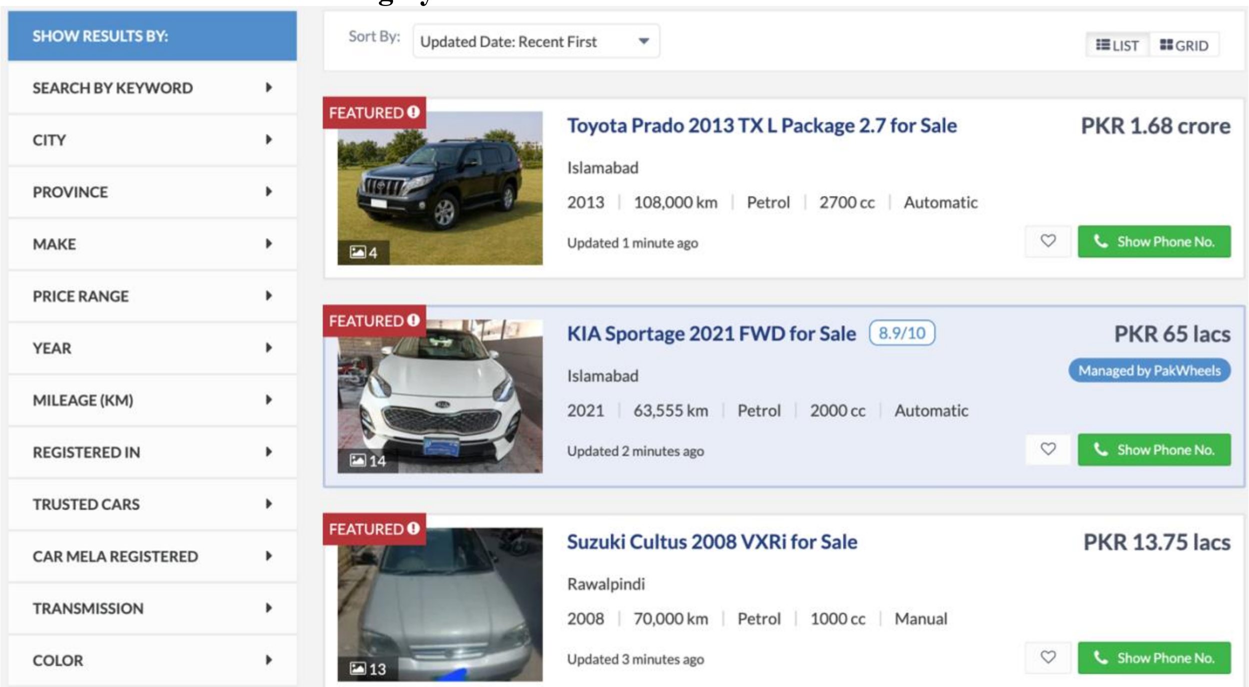
6. Class Diagram



◆ filled diamond = Composition
◇ open diamond = Aggregation
▷ open arrow = Inheritance ↑

7. PakWheels Features

Feature 01: Vehicle Filtering System




```cpp
void filterByType(string t) const {
    bool found = false;
    for (int i = 0; i < count; i++) {
        if (listings[i]->isApproved() && listings[i]->getVehicle() != nullptr) {
            if (listings[i]->getVehicle()->getType() == t) {
                listings[i]->display();
                found = true;
            }
        }
    }
    if (!found)
        cout << "No listings found for type: " << t << endl;
}
```

```
void filterByKeyword(string key) const {
    bool found = false;
    for (int i = 0; i < count; i++) {
        if (listings[i]->isApproved() && listings[i]->getVehicle() != nullptr) {
            if (listings[i]->getVehicle()->matchesFilter(key)) {
                listings[i]->display();
                found = true;
            }
        }
    }
    if (!found)
        cout << "No results for: " << key << endl;
}
```

```
void filterByPrice(float minP, float maxP) const {
    bool found = false;
    for (int i = 0; i < count; i++) {
        if (listings[i]->isApproved() && listings[i]->getVehicle() != nullptr) {
            float p = listings[i]->getVehicle()->getPrice();
            if (p >= minP && p <= maxP) {
                listings[i]->display();
                found = true;
            }
        }
    }
    if (!found)
        cout << "No listings in that price range.\n";
}
```

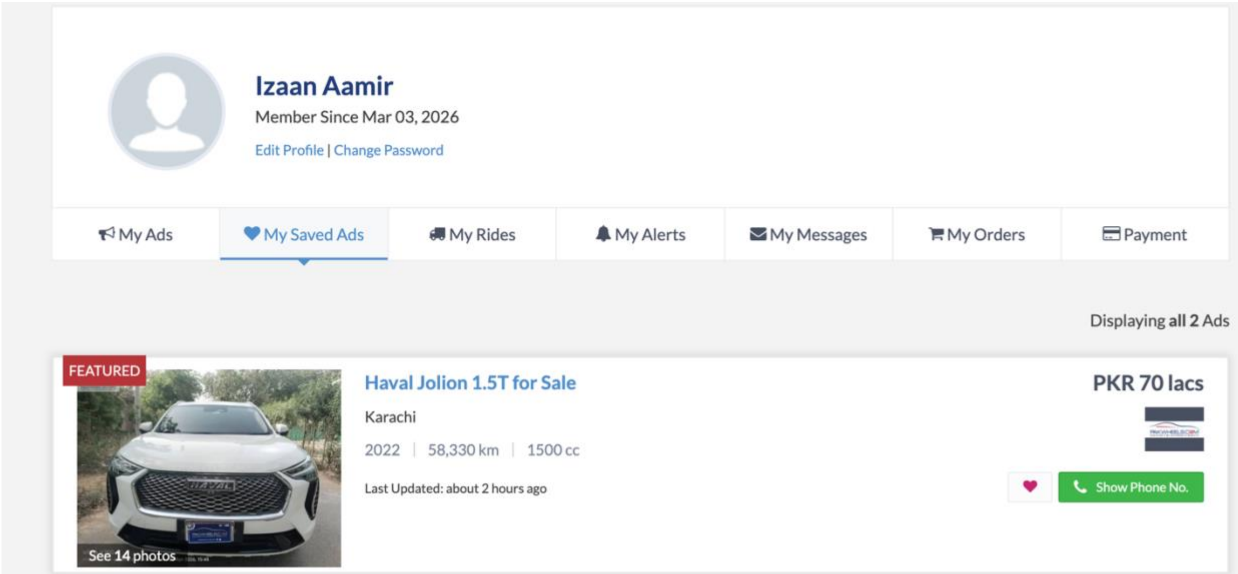
Reasoning:

PakWheels allows users to filter vehicles based on category, brand, model, and price range.

Similarly, the Marketplace class contains multiple filtering functions which display only approved listings matching the search criteria.

Polymorphism is also used because different vehicle types override display functions.

Feature 02: Favorites System




```cpp
void addFavorite(Listing* l) {
    for (int i = 0; i < favCount; i++) {
        if (favorites[i]->getId() == l->getId()) {
            cout << "Already in favorites.\n";
            return;
        }
    }
    if (favCount < MAX_FAVORITES) {
        favorites[favCount++] = l;
        cout << "Added to favorites.\n";
    }
}

void viewFavorites() const {
    if (favCount == 0) {
        cout << "No favorites saved.\n";
        return;
    }
    for (int i = 0; i < favCount; i++)
        favorites[i]->display();
}
```

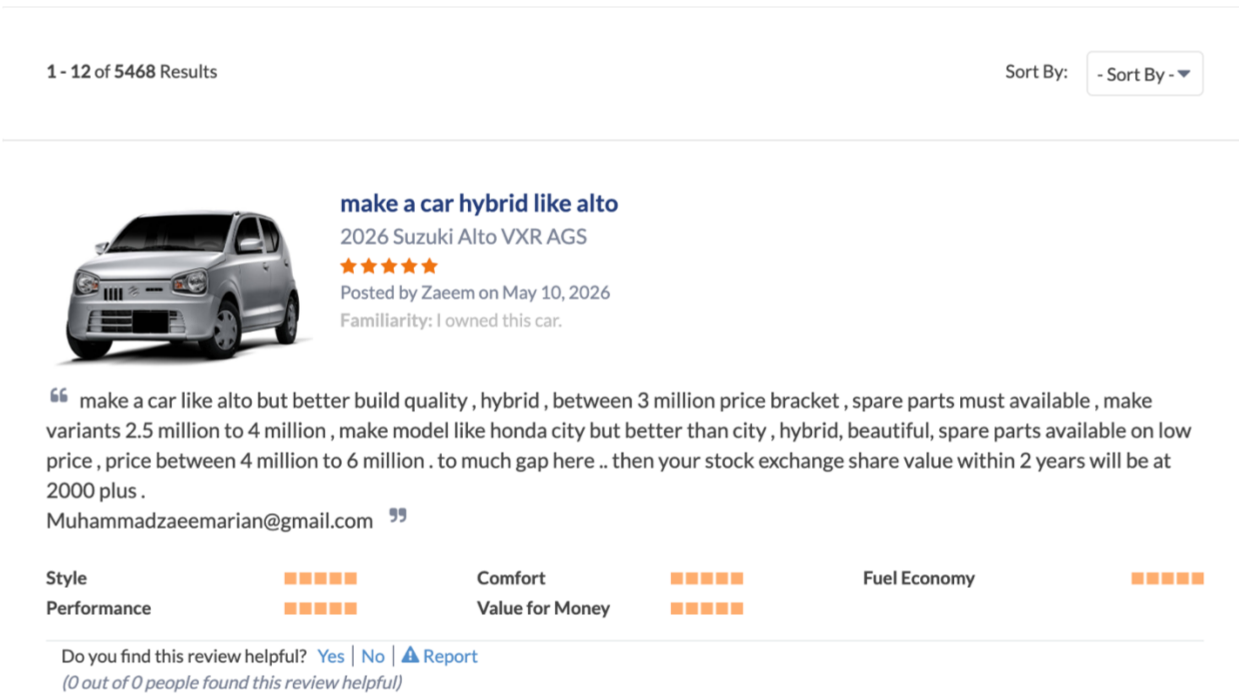
Reasoning:

Buyers can save vehicle listings to favorites for future viewing.

Aggregation is used because Buyer stores references to Listing objects without owning them.

Duplicate favorites are prevented using listing ID comparison.

Feature 03: Seller Reviews & Ratings




```
void showReviews() const {
    for (int i = 0; i < reviewCount; i++)
        reviews[i].display();
}
```

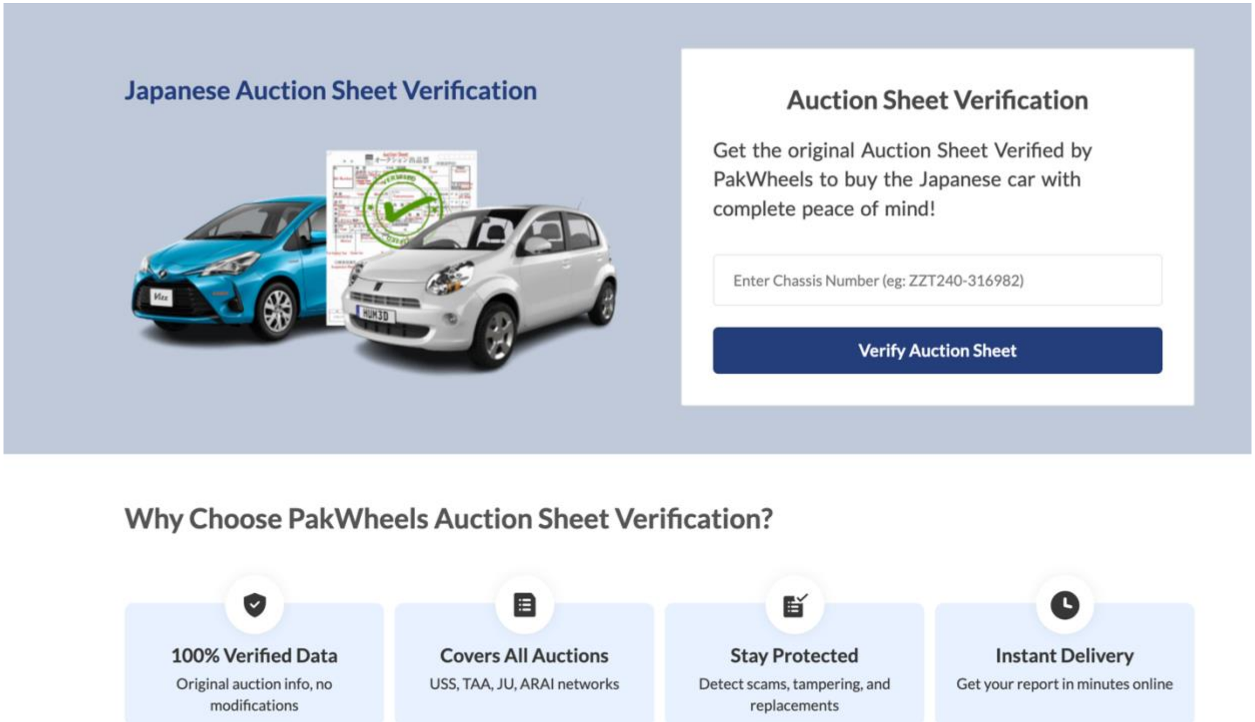
```
class Review {
private:
    int raterId;
    int stars;
    string comment;

public:
    Review() {
        raterId = 0;
        stars = 0;
        comment = "";
    }

    Review(int rid, int s, string c) {
        raterId = rid;
        stars = s;
        comment = c;
    }

    bool operator==(const Review& r) const {
        return raterId == r.raterId;
    }

    int getStars() const {
        return stars;
    }

    void display() const {
        cout << "Rater ID: " << raterId << " | Stars: " << stars << "/5 | " << comment << endl;
    }

    friend float calcAvgRating(Review arr[], int count);
};
```

Reasoning:

The system allows buyers to leave reviews and ratings for sellers.

A friend function calcAvgRating() calculates average seller ratings using arrays of Review objects.

Feature 04: Bidding System




```cpp
bool operator>(const Bid& b) const {
    return amount > b.amount;
}

bool operator==(const Bid& b) const {
    return bidderId == b.bidderId;
}
```

```
    void placeBid(int buyerId, float amt) {
        if (!approved) {
            cout << "Listing not approved yet.\n";
            return;
        }
        if (bidCount >= MAX_BIDS) {
            cout << "Bid limit reached.\n";
            return;
        }
        Bid newBid(buyerId, amt);
        for (int i = 0; i < bidCount; i++) {
            if (bids[i] == newBid) {
                cout << "You already placed a bid.\n";
                return;
            }
        }
        bids[bidCount++] = newBid;
        cout << "Bid placed: Rs." << amt << endl;
    }

    void showBids() const {
        if (bidCount == 0) {
            cout << "No bids yet.\n";
            return;
        }
        for (int i = 0; i < bidCount; i++)
            bids[i].display();
    }
```

Reasoning:

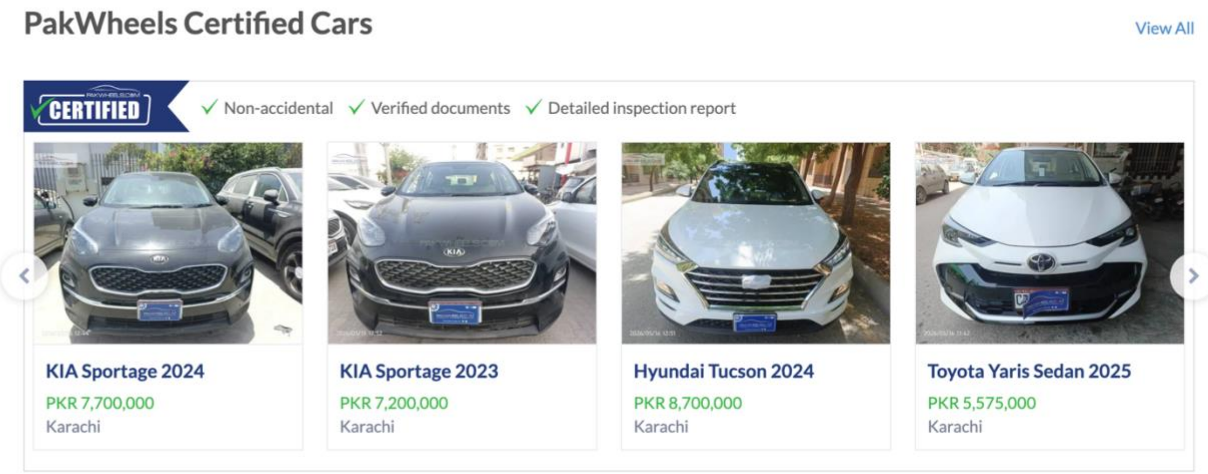
The system allows buyers to place bids on approved listings.

The Bid class stores:

- Bidder ID
- Bid amount

Operator overloading (>) is used to compare bids and determine the highest bid.

Feature 05: Vehicle Approval System




```cpp
void approveListing(Listing& l) {
    l.approve();
    cout << "Listing " << l.getId() << " approved.\n";
}
```

Reasoning:

Admin approval is required before listings become publicly visible.

This ensures platform authenticity and prevents invalid listings.